Теоретическая информатика. Домашка 3. Анти-галлюцинатор, часть I

Екатерина Шеметова (katyacyfra@gmail.com)

17.09.2026

Мягкий дедлайн: 01.10.2026 23:59
Жесткий дедлайн: 08.10.2026 23:59
Количество баллов на зачет: 12

Современные LLM отлично понимают текстовые описания задач, но часто галлюцинируют и совершают синтаксические или логические ошибки при генерации сложных регулярных выражений или автоматов.

Ваша задача статически проверить качество генерации автоматов и регулярок, а также сравнить выбранные модели по разным критериям.

1 Входные данные

Выберите и зафиксируйте не менее трёх LLM (это могут быть либо разные модели, либо одна и та же модель с разными параметрами).

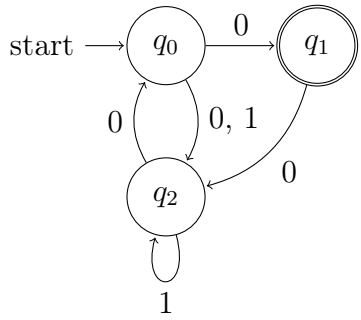
Реализуйте функцию отправки промпта к API моделей.

Вход: Текстовое описание формата (например: «ID сотрудника: начинается строго с двух букв 'ID', затем следует от 1 до 3 цифр, но число не может начинаться с нуля»).

Выход: Конечный автомат, сгенерерированный LLM, в следущем формате (пишем в файлик):

`n` //число состояний
`m` // размер алфавита
`0 2 3` // номера начальных состояний через пробел
`0 2 3` // номера конечных состояний через пробел
`2 2 4` // переход по символу "2" из состояния 2 в состояние 4
...
`3 0 1` // переход по символу "0" из состояния 3 в состояние 1

Пример входа для автомата:




```
3
2
0
1
0 0 1
0 0 2
0 1 2
1 0 2
2 1 2
2 0 0
```

1.1 Тесткейсы, на которых будем проверять

Описание передаем ровно в том виде, в каком оно в табличке! Математические значки в intersection запишите в tex-формате.

ID	Описание
basic	число ноликов чётно, число единиц нечётно, нет подстрок 00 и нет подстрок 11, алфавит {0 , 1}
4th-symbol	четвёртый символ с конца равен 1, алфавит {0 , 1}
employer	ID сотрудника: начинается с 3 букв региона, потом 4 цифры, причем если регион SPB, то цифры не могут начинаться с нуля, алфавит для простоты возьмем такой: {A, M, C, S, P, B, 0, 1, 2, 3, 4, 5}
intersection	$L = (A)^R \cap B$, где $A = \{w \in \{0,1\}^*,$ третий символ с начала равен $1\}$, $B = \{w \in \{0,1\}^*,$ w содержит ровно один $0\}$
personal	придумайте минимум 1 свой пример

1.2 Что делаем?

Делаем запрос к модели, просим её построить конечный автомат в формате выше для тесткейса.

Один запрос, один тесткейс.

По каждому тесткейсу делать не менее 50 запросов, потом нужно будет посчитать всякие статистики по ним.

2 Проверки/метрики

Начинаем наслаждаться знаниями, полученными на ТИ :)

Пишем на любимом языке программирования!

Не забываем про тесты!

(3 балла, Корректность) Реализуйте проверку корректности формата сгенерированного автомата: соответствие формату, нет ли ошибочных переходов (например, из несуществующих состояний и т.д), корректно ли число состояний и размер алфавита и т.д.

(5 балла, Симулятор) Реализуйте проверку корректности сгенерированного автомата: напишите метод, который принимает файлик со сгенерированным автоматом и строку, а возвращает true|false (принимается ли автоматом данная строка). Добавьте метод-проверку, который симулирует автомат на строках, принадлежащих и не принадлежащих языку из тесткейса (строк должно быть несколько, чем больше, тем лучше). На основании проверки делайте вывод о корректности сгенеренного автомата (true|false).

(7 баллов, NFA → DFA) Реализуйте преобразование NFA в DFA с помощью построения подмножеств *Не плодите сразу экспоненту состояний, идите от стартового состояния, сливая по дороге состояния, если нужно!*. Выход: число состояний в полученном детерминированном автомате

3 Финальная табличка с метриками и глубокомысленными выводами (без неё не зачту)

Каждый сгенеренный автомат должен быть подвергнут проверкам из пункта 2. В итоге получаем табличку следующего вида:

Потом анализируем табличку и делаем глубокомысленные выводы!

	Корректные по формату автоматы, %			Детерминированные автоматы, %			Среднее количество состояний в DFA			Корректные по симуляции автоматы, %		
	GPT-6	ГигаЧат	...	GPT-6	...	...	GPT-6	...	...	GPT-6	...	...
basic	100%	121%	...	100%	100%	100%	20	100500	2			
4th-symbol	12%	...		5%								
employer	10%											
...												
Всего	20%											